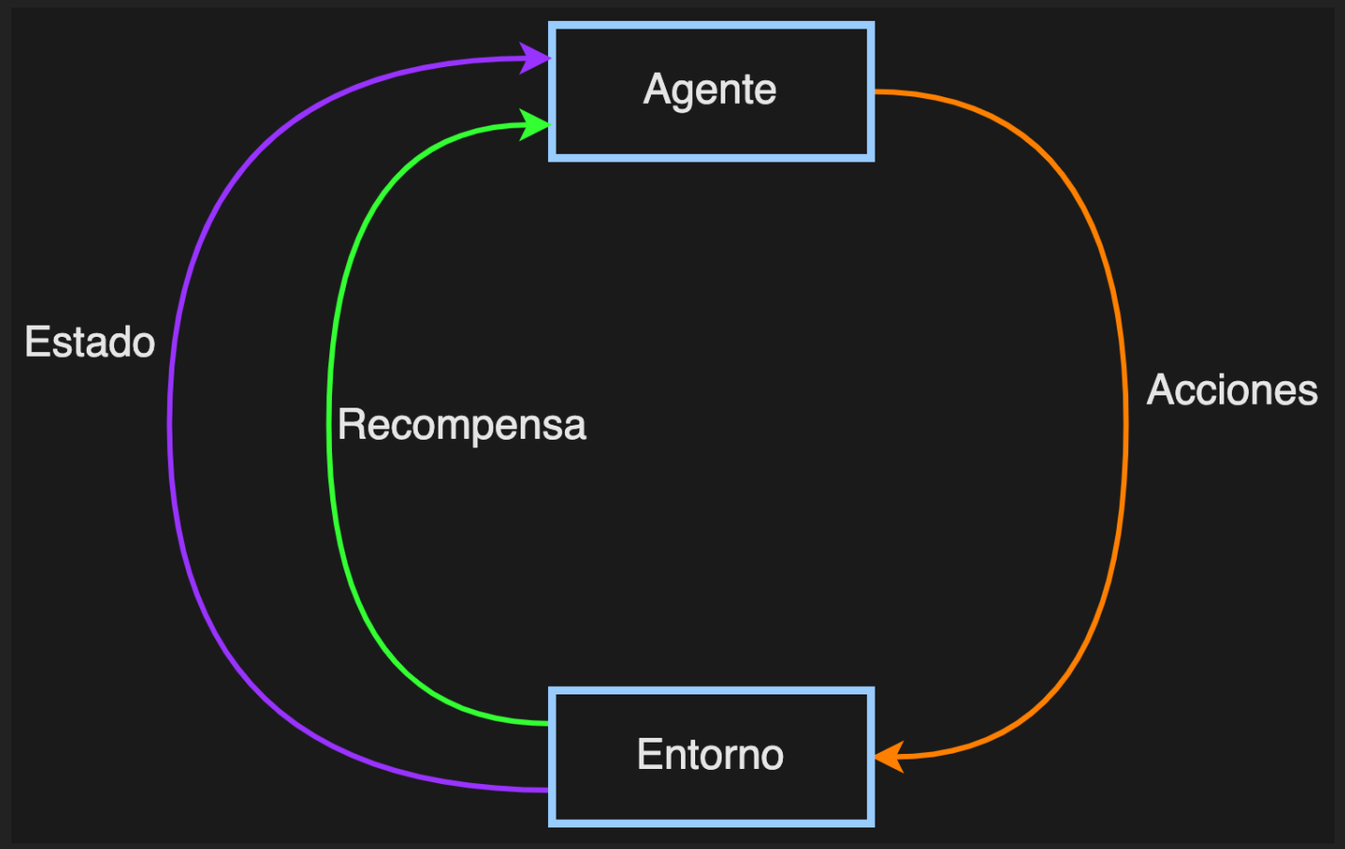
11

MÉTODOS DE GRADIENTE DE POLÍTICA

CÓMO FUNCIONAN LOS MÉTODOS PG?

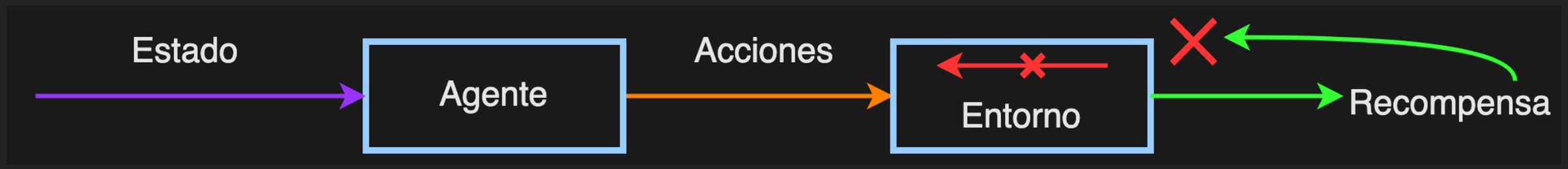
- Consideremos la estructura básica de un esquema RL:



- El entorno puede pensarse como una caja negra que procesa acciones, produce un nuevo estado y una recompensa.

CÓMO FUNCIONAN LOS MÉTODOS PG?

- Si se analiza el esquema como un sistema abierto y auto-retroalimentado, el problema es que el entorno:
  - ✓ Realiza procesos para los cuales no tenemos una expresión o función.
  - ✓ No es diferenciable. No se puede calcular su derivada.
- Como no es posible calcular su derivada tampoco es posible retropropagar valores a través de él.



CÓMO FUNCIONAN LOS MÉTODOS PG?

- Esto es un problema porque el agente debe aprender (se puede considerar que el agente internamente tiene una red neuronal con pesos que deben ser actualizados).

CÓMO FUNCIONAN LOS MÉTODOS PG?

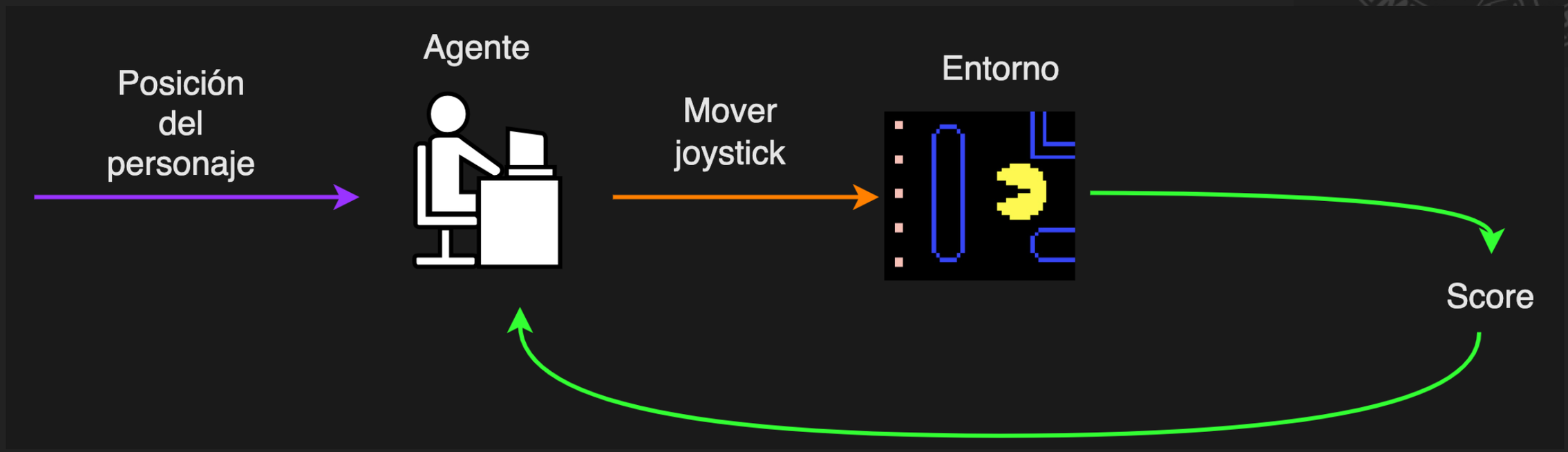
- Para actualizar los pesos se necesita retropropagar a través de la red neuronal. Motivo por el cual se necesita el gradiente a la salida de la red.

CÓMO FUNCIONAN LOS MÉTODOS PG?

- En lugar de usar la salida del entorno para intentar retropropagar a través de él y luego tratar de retropropagar a través del agente para actualizar los pesos de la red, se debería encontrar otras formas de actualizar los pesos.

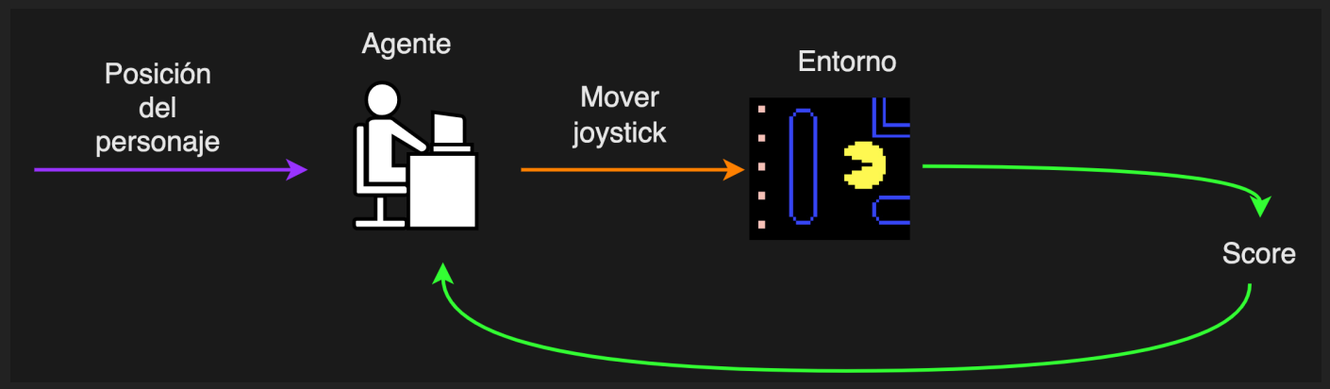
CÓMO FUNCIONAN LOS MÉTODOS PG?

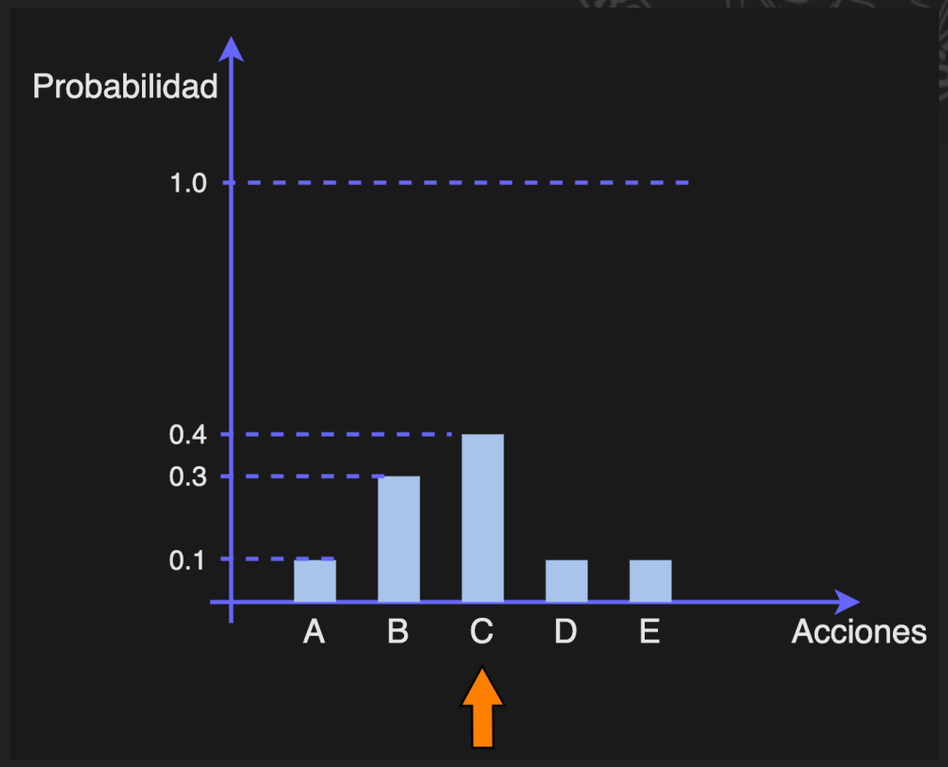
- Imaginemos una persona intentando aprender a ganar en un videojuego.
- La persona realizará una acción en el juego que le otorga un puntaje positivo y por eso la persona tenderá a recordar esta acción y hará esa acción más a menudo.
- La persona usa el puntaje recibido para reforzar aquellas acciones que el entorno considera buenas.



CÓMO FUNCIONAN LOS MÉTODOS PG?

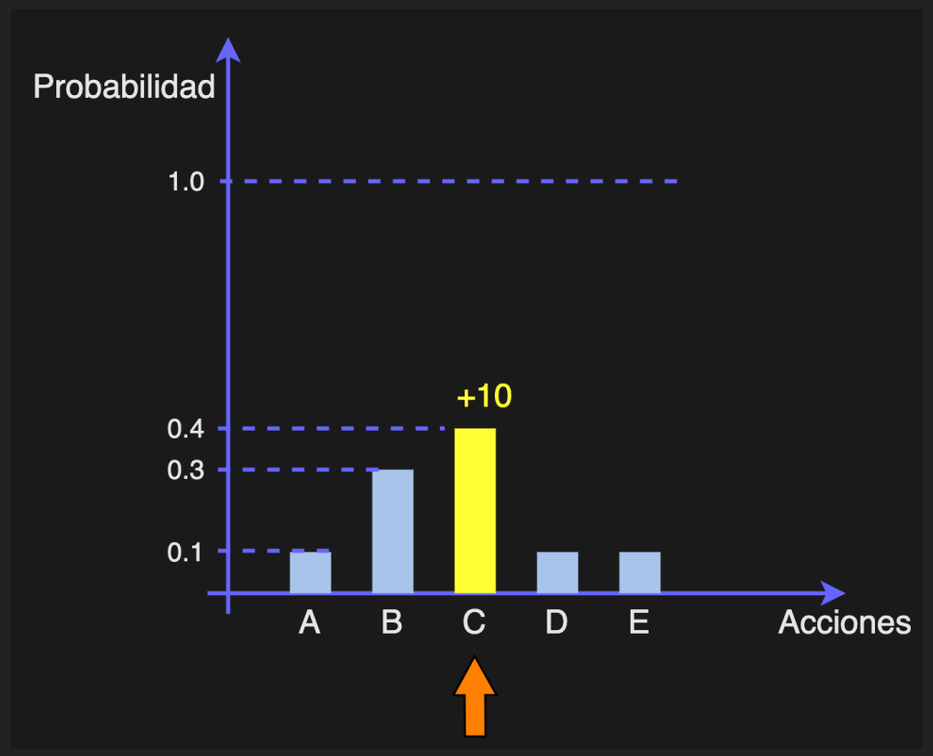
- Es decir, si la persona selecciona una acción que redunda en un buen resultado, entonces debería tomar esa acción más a menudo en el futuro.
- Se podría pensar que existe una distribución de acciones posibles que puede tomar la persona mientras juega.





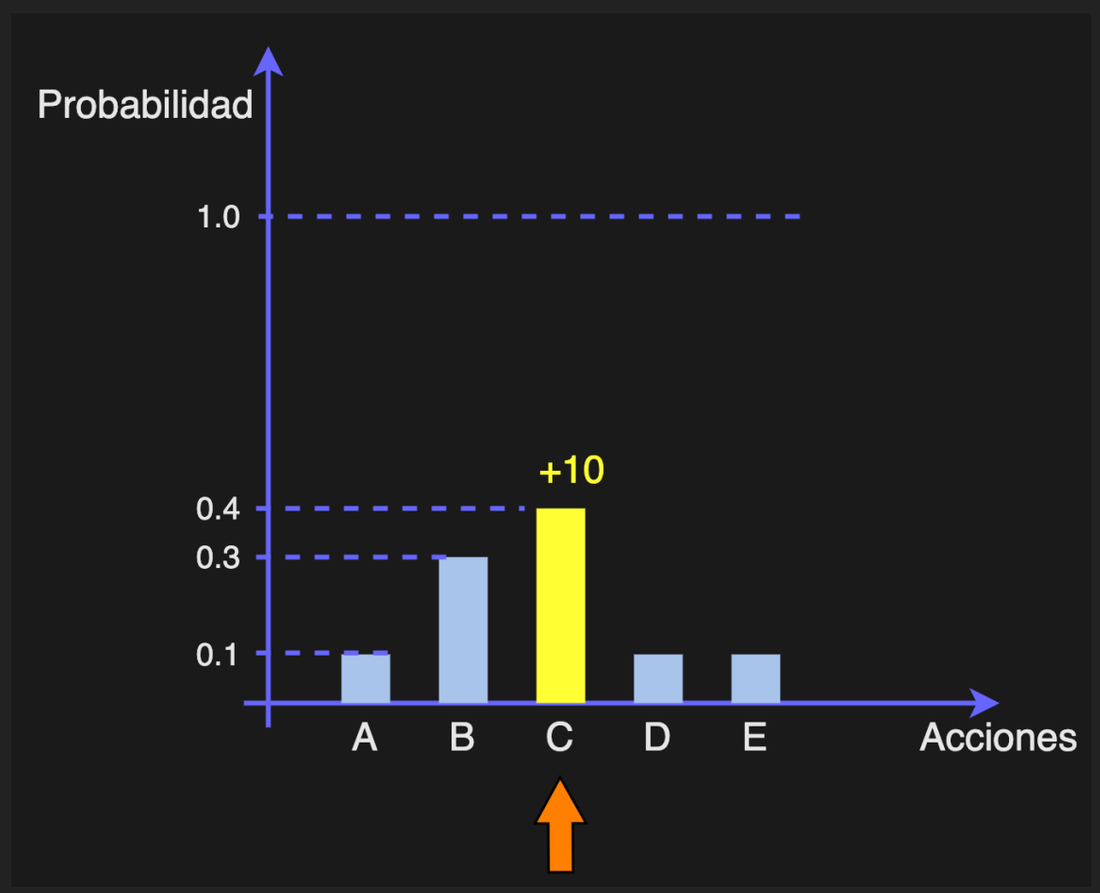
CÓMO FUNCIONAN LOS MÉTODOS PG?

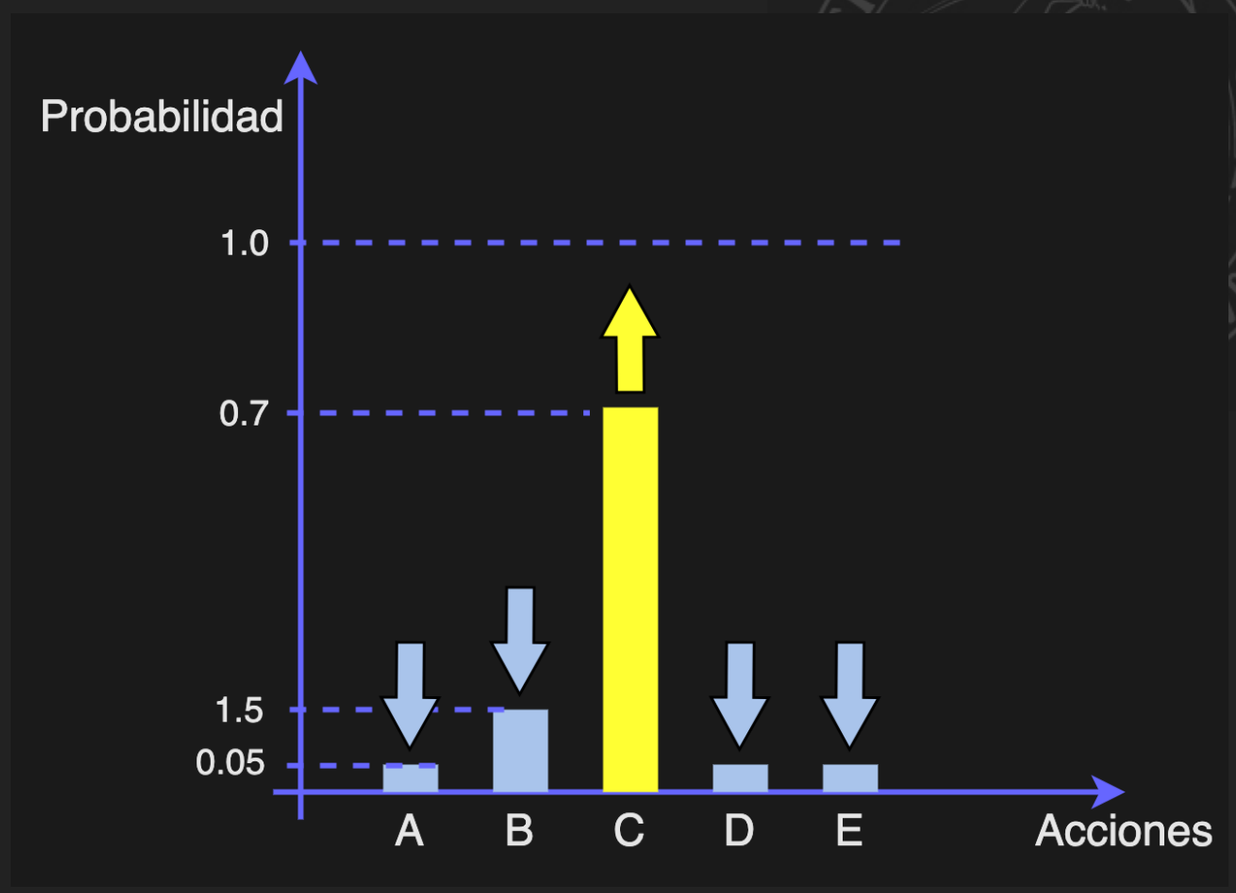
- Si en esta distribución de acciones posibles una de estas acciones resulta llevar a un buen resultado (devuelve una recompensa grande) entonces habría que "aplastar" lateralmente la distribución alrededor de esa acción, de modo que esa acción ocurra más a menudo.



CÓMO FUNCIONAN LOS MÉTODOS PG?

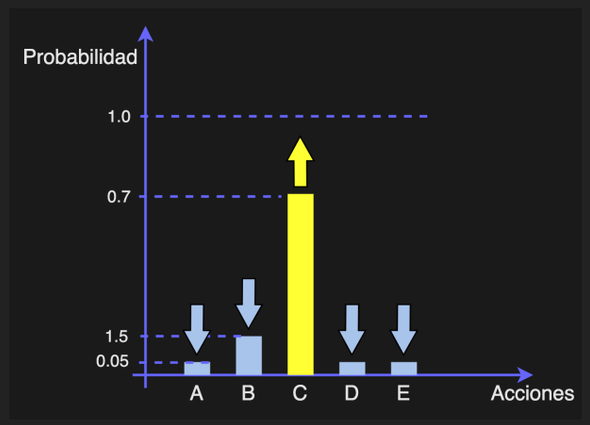
- La acción C ahora tiene mayor probabilidad dado que la "empujamos" hacia arriba y bajamos las demás.

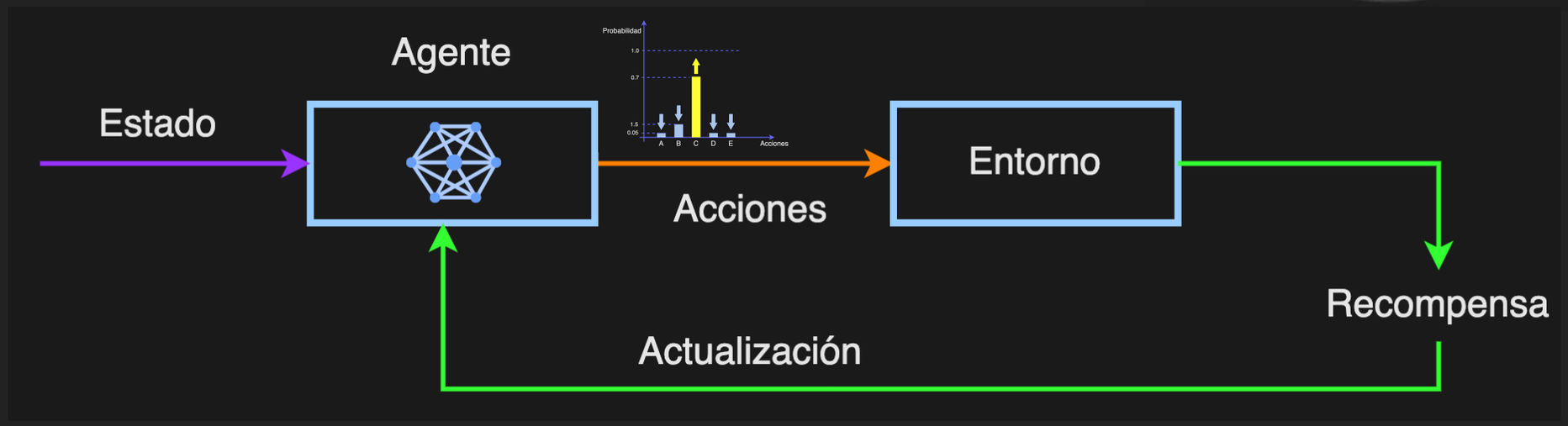




CÓMO FUNCIONAN LOS MÉTODOS PG?

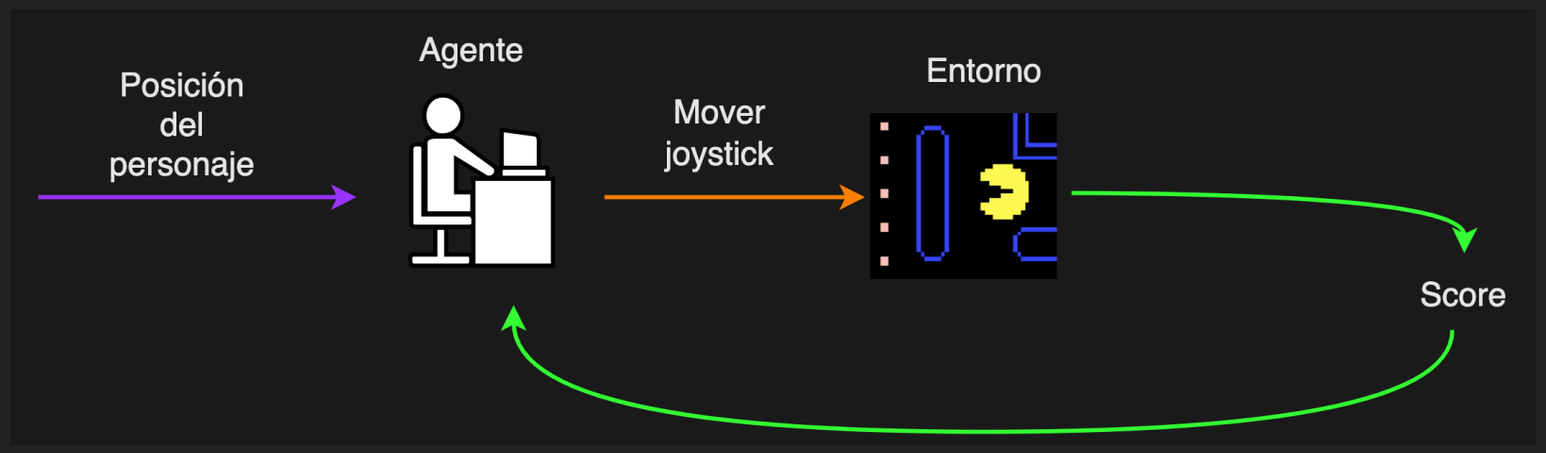
- De forma más general, se puede usar un resultado bueno o malo como señal para el aprendizaje.
- Esto se puede usar siempre que exista alguna medida de 'bondad' a la salida de la red neuronal.
- Cuando no se puede retropropagar o diferenciar a través del proceso que entrega la salida, en lugar de usar el gradiente descendente como se hace normalmente en aprendizaje supervisado, se puede ajustar el gradiente ascendente del comportamiento (la política) en la misma dirección de la recompensa.
- Esta es la idea de los Métodos de Gradiente de Política. Es decir, si se tiene algo que no se puede diferenciar, se usa la recompensa como señal para ayudar a cambiar los pesos.





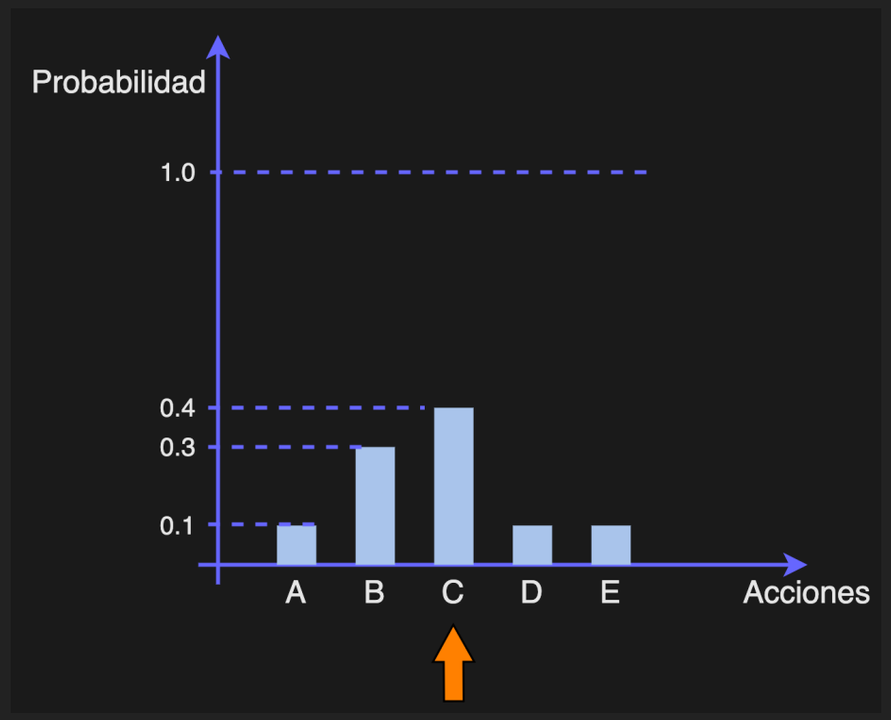
CONCEPTO GENERAL DE PG

- Los métodos de aprendizaje como SARSA, Q-Learning y DQN utilizan funciones de valor V(s) o Q(s, a).
- En los métodos de gradiente de política (Policy Gradient), se aproxima directamente la política óptima sin necesidad de usar una función de valor.
- Ya no se considera el valor de las acciones para que el agente aprenda, sino que el agente actuará según una política que se irá optimizando gradualmente.



CONCEPTO GENERAL DE PG

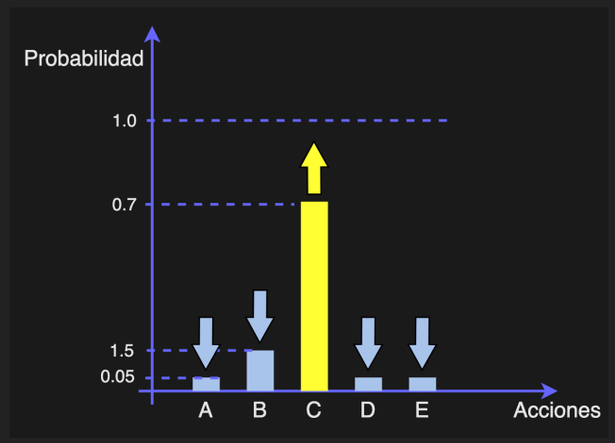
- En los métodos de valor de acción (como Q-Learning), el agente aprende qué tan buena es cada acción a en un estado s particular. Estima ese valor. Luego, basándose en esos valores aprendidos, deduce una política (una forma de comportarse).
- Los métodos de gradiente de la política aprenden una política parametrizada por θ (los pesos de una red neuronal). Esto es, se comportan como una función que mapea estados a probabilidades de acciones. Aprenden la estrategia en sí, no solo el valor de las partes de la estrategia.



CONCEPTO GENERAL DE PG

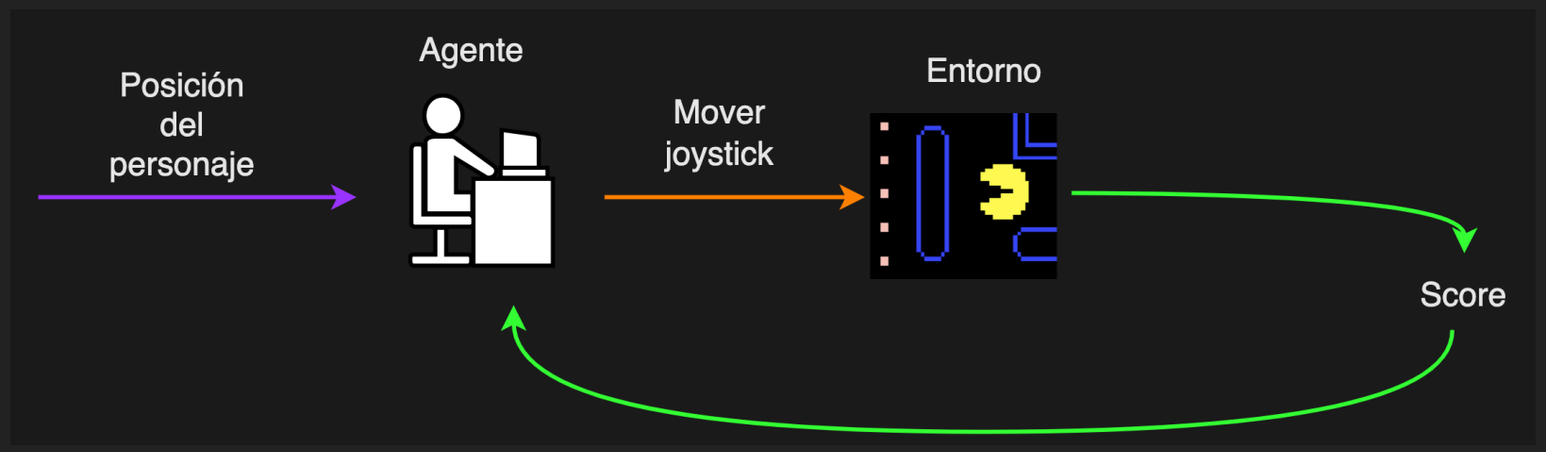
- Estos métodos usan parámetros, tales como θ (theta), para definir la política. Ejemplo: la notación $\pi(a|s, \theta)$, es la probabilidad de tomar la acción 'a' en el estado 's' y dados esos parámetros de política θ.
- Se llaman métodos de gradiente de política porque aprenden los parámetros de la política siguiendo el gradiente (la dirección de mayor aumento de alguna medida de rendimiento). Esta medida de rendimiento se denomina J(θ).
- Es un ascenso por gradiente, intentando maximizar el rendimiento. La actualización de los pesos implica tomar los parámetros antiguos θ y sumarles un tamaño de paso α multiplicado por el gradiente de J(θ).

$$\theta_{\{t+1\}} = \theta_t + \alpha \nabla J(\theta_t)$$



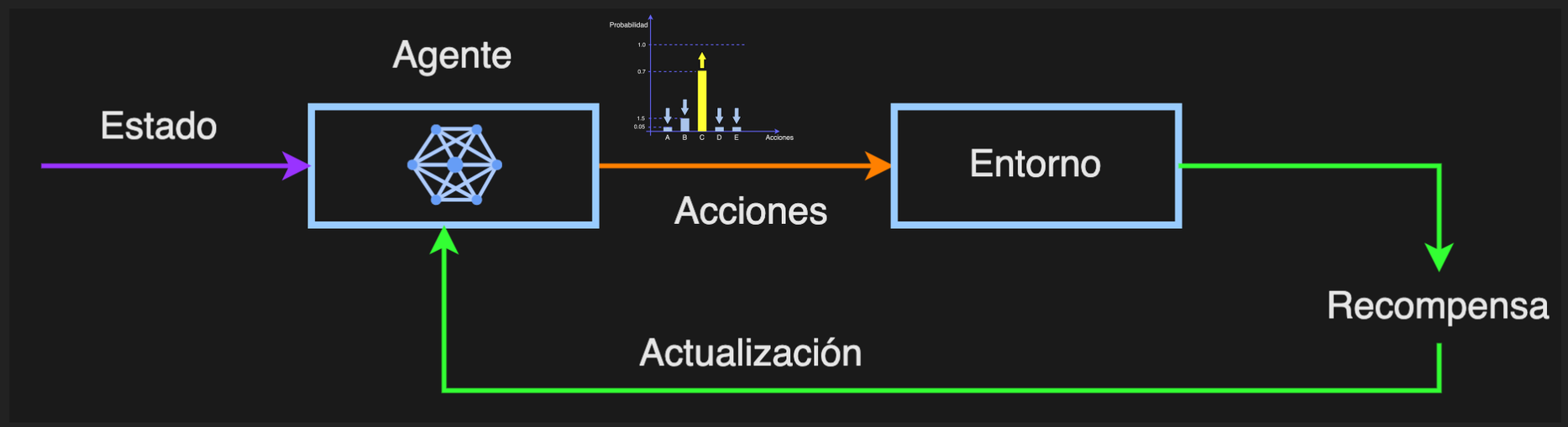
GRADIENTES DE POLÍTICA

- Otra forma de que un agente aprenda, en lugar de usar una función de valor, es expresar la política directamente sin necesidad de una función de valor.
- En general los métodos PG (Policy Gradient), la política es independiente de la función de valor (es así en métodos REINFORCE, no en Actor-Critic).
- El agente aprende la política π_θ, la cual indica qué acción tomar en un estado s, sin requerir una función de valor.



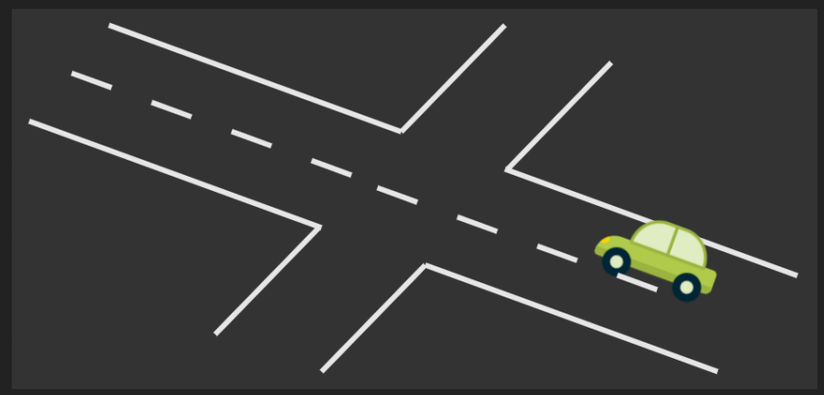
GRADIENTES DE POLÍTICA

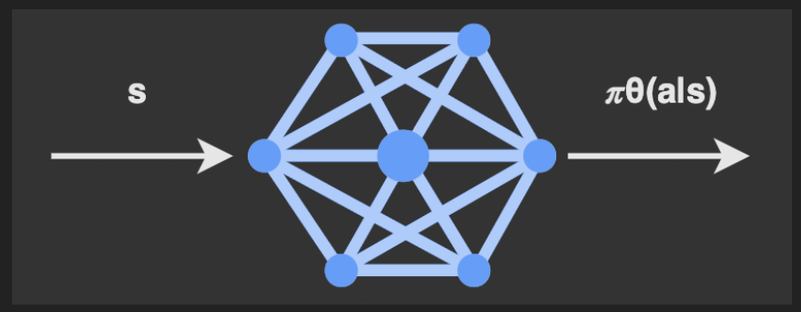
- Al igual que en Q-learning, el objetivo sigue siendo obtener la máxima recompensa, con la diferencia de que ahora el agente sigue una política parametrizada que depende de θ.
- La probabilidad de tomar cada acción para un determinado estado la genera una red neuronal.
- En cada interacción del agente en el entorno la red neuronal:
  - ✓ Tiene a su entrada el estado actual,
  - ✓ Ajusta sus pesos θ para que las mejores acciones tengan mas probabilidad de ser tomadas y,
  - ✓ Entrega en su salida una distribución de probabilidad de las acciones.

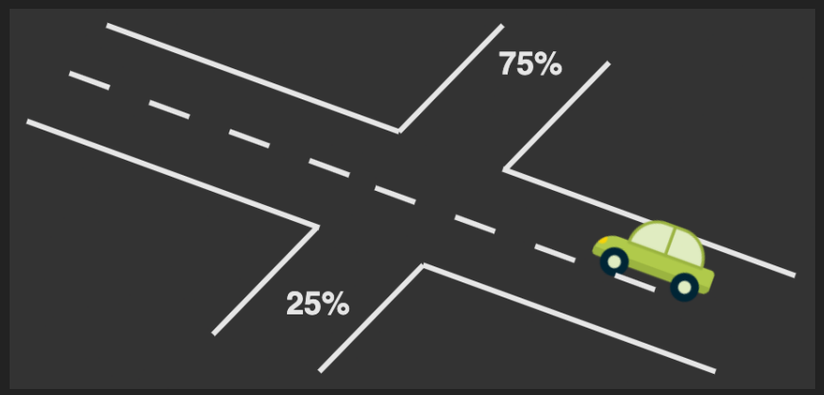


EJEMPLO: CONDUCCIÓN DE UN COCHE

- El vehículo puede moverse a la derecha o a la izquierda, pero doblar a la izquierda recibe menos recompensa.
- Para cada estado, la política π_θ asignará más probabilidad a la mejor acción.
- En cada estado s se produce una salida de la red hasta que finalice el episodio.
- Al final del episodio se actualizan los θ de la red (a diferencia de los métodos de función de valor en los que las actualizaciones se producían luego de cada acción)





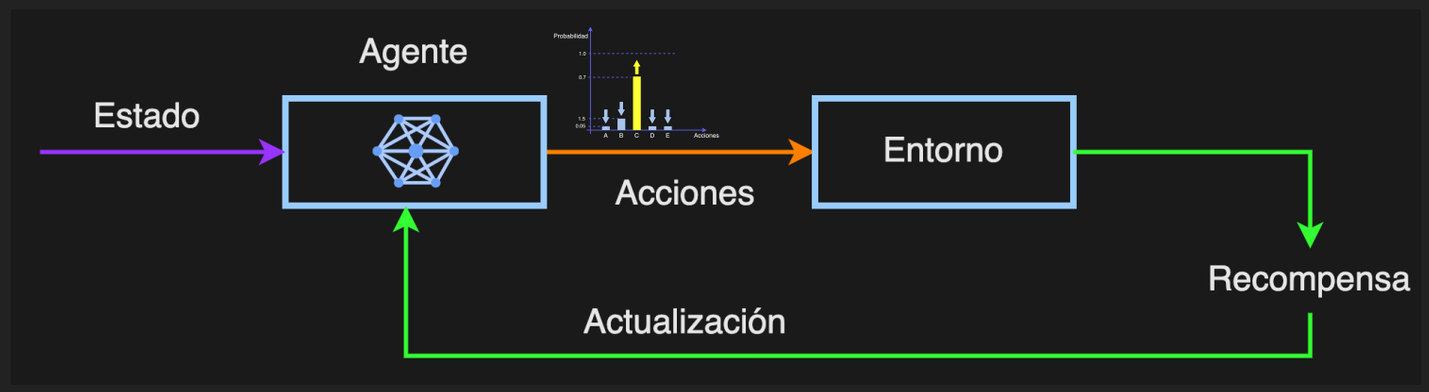


EJEMPLO: CONDUCCIÓN DE UN COCHE

- La red deja que el agente tome acciones libremente pero en cada episodio se ajusta y otorga al agente probabilidades de acciones cada vez mejores.
- Durante el episodio, la red genera acciones libremente, pero al final del episodio se actualizan los parámetros θ. Esto es típico en REINFORCE.
- En cada nuevo episodio, la red ajusta sus parámetros para dar mejores probabilidades de acción.
- La política se parametriza así:

$$\pi(a|s, \theta) = P(A_t = a \mid S_t = s, \theta_t = \theta)$$

se lee, la probabalidad de que el agente tome la acción a en el tiempo t dado un estado s en el mismo tiempo t y dado el vector de parámetros θ.



ESPACIO DE LAS ACCIONES

- El espacio de acciones bajo la distribución de probabilidad parametrizada por θ puede ser:

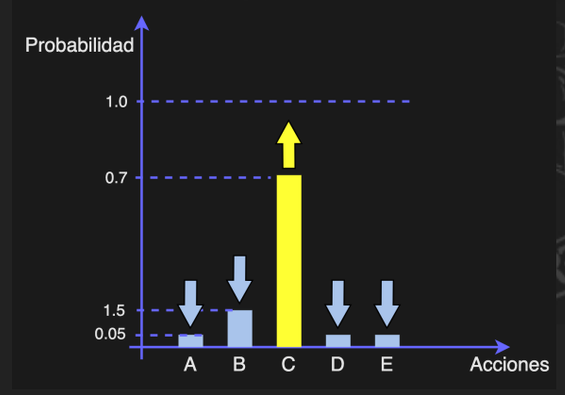
  ✓ Discreto: la distribución de probabilidad es categórica*, y la salida suele pasar por una función softmax**.

  *Una distribución categórica es una distribución de probabilidad que se usa cuando se tienen varias clases o categorías posibles, y solo una de ellas puede ocurrir a la vez.

  **La función softmax es una función matemática que convierte un vector de números reales (por ejemplo, puntuaciones o logits) en una distribución de probabilidad.

  Ejemplo: Dado un vector $Z=[z_1, z_2, \ldots z_n]$ la funcon softmax devuelve:

  $$\text{Softmax}(z_i) = e^{z_i} / \text{sum}(e^{z_j})$$



  La función softmax:

  ✓ Realza el valor más grande y suprime los pequeños.

  ✓ Da más probabilidad al valor más alto, pero aún mantiene un "reparto" suave entre todos.

  ✓ Se usa para representar la probabilidad de que una clase sea la correcta, especialmente en problemas de clasificación multiclase (como MNIST, CIFAR-10, etc.).

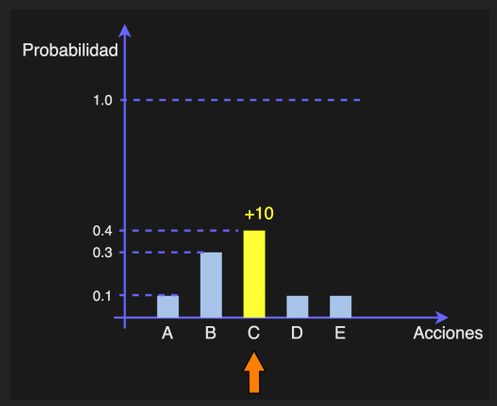
  ✓ Continuo: la salida está definida por los parámetros de una distribución continua (por ejemplo, la media y la varianza de una distribución gaussiana).

TIPOS DE POLÍTICAS

- Determinista: para un estado s, la política $\pi(s)$ devuelve directamente la acción a que maximiza la recompensa.
- Estocástica: para un estado s, la política $\pi(a|s)$ devuelve una distribución de probabilidad continua del conjunto de acciones a tomar. Es decir que:

$$\pi(a|s, \theta) = P(A_t = a \mid S_t = s, \theta_t = \theta)$$

- Esta distribución continua de probabilidad permite que una política estocástica se aplique en espacios continuos.
- El objetivo de un algoritmo PG es ajustar una red neuronal parametrizada por θ, para que aprenda una política estocástica $\pi(a|s, \theta)$, representada mediante una distribución de probabilidad que mapea estados con acciones, buscando maximizar una función objetivo $J(\theta)$.



FUNCIÓN OBJETIVO J(θ)

- Caso discreto:

Policy Gradient

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T} \gamma^t r(s_t, a_t)\right]$$

Monte Carlo

$$v_\pi(s) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s\right]$$

donde $J(\theta) = v_{\pi\theta}(S_0)$ es el valor esperado de las recompensas descontadas para la trayectoria completa τ (tau) desde el estado inicial S_0 y siguiendo una política π_θ.

En PG no interesa el valor de un estado en particular, pero sí interesa el rendimiento promedio de toda la política.

- Caso continuo o de horizonte infinito (PG en entornos continuos sin episodios y sin descuento):

$$J(\theta) = \lim_{h \to \infty} \frac{1}{h} \sum_{t=0}^{h} \mathbb{E}[r(s_t, a_t) \mid A_{0:t} \sim \pi_\theta]$$

$$\lim_{t \to \infty} \mathbb{E}[r(s_t, a_t) \mid A_{0:t} \sim \pi_\theta]$$

- $J(\theta)$=promedio de recompensas por paso a medida que $t \to \infty$

ACTUALIZACIÓN DE θ

- En Deep Learning se minimiza una función de pérdida. En Policy Gradient se maximiza el rendimiento J(θ) mediante ascenso por gradiente:

$$\theta_{\{t+1\}} = \theta_t + \alpha \nabla J(\theta_t)$$

donde $\nabla J(\theta_t) \in \mathbb{R}^{d'}$ es una estimación estocástica y α es el coeficiente de aprendizaje. Es decir que el gradiente de J es un vector de la misma dimensión que θ.

- La actualización y optimización de θ se hace en la misma dirección del gradiente mediante el ascenso del gradiente.

TEOREMA DE GRADIENTE DE POLÍTICA

- Vimos anteriormente que la función objetivo en el caso discreto era:

$$J(\theta) = v_{\pi\theta}(S_0)$$

donde $v_{\pi\theta}(S_0)$ es el valor de la función para π_θ y S_0 es el estado inicial.

- Si se maximiza $J(\theta)$ se está maximizando $v_{\pi\theta}$.
- También vimos que la actualización es:

$$\theta_{\{t+1\}} = \theta_t + \alpha \nabla J(\theta_t)$$

entonces:

$$\nabla J(\theta_t) = \nabla v_{\pi\theta}(S_0)$$

TEOREMA DE GRADIENTE DE POLÍTICA

- Como a y s dependen del entorno (desconocido), no se puede calcular directamente el $\nabla v_{\pi\theta}(S_0)$.
- La solución a esto es aplicar el teorema de gradientes de politica que expresa el rendimiento J respecto a la politica (Sutton et al., 2018):

$$\nabla J(\theta) = \sum_s \mu(s) \sum_a q_\pi(s, a) \nabla \pi(a|s, \theta)$$

donde:

✓ $\mu(s)$: distribución de estados bajo la política π

✓ $q_\pi(s, a)$: valor de acción de la política

✓ $\nabla \pi(a|s, \theta)$: gradiente de la política

- Si se busca mejorar la política se debe cambiar θ en la dirección donde más aumentan las probabilidades de tomar acciones buenas (las que tienen alto $q_\pi(s,a)$)
- Este teorema es model-free (no requiere conocer el modelo del entorno).

TEOREMA DE GRADIENTE DE POLÍTICA

- No conozco el gradiente de π, es decir, $\nabla\pi(a|s, \theta)$, pero lo puedo escribir como:

$$\nabla_\theta \pi(a|s,\theta) = \pi(a|s,\theta) \cdot \frac{\nabla_\theta \pi(a|s,\theta)}{\pi(a|s,\theta)} = \pi(a|s,\theta) \cdot \nabla_\theta \ln \pi(a|s,\theta)$$

reemplazando el gradiente de pi:

$$\nabla J(\theta) = \sum_s \mu(s) \sum_a q_\pi(s,a)\pi(a|s,\theta)\nabla_\theta \ln \pi(a|s,\theta)$$

Expresado en términos de la esperanza:

$$\nabla J(\theta) = \mathbb{E}_{s\sim\mu, a\sim\pi_\theta}\left[q_\pi(s,a)\nabla_\theta \ln \pi(a|s,\theta)\right]$$

Sabemos que:

$$q_\pi(s,a) = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,\middle|\, S_t = s, A_t = a\right]$$

luego Gt es la suma de recompensas descontadas:

$$R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \qquad \mathbb{E}_\pi\left[G_t \nabla_\theta \ln \pi(A_t|S_t,\theta)\right]$$

Finalmente reemplazo el gradiente de J en la actualización theta:

$$\nabla J(\theta) = \mathbb{E}_\pi\left[G_t \nabla_\theta \ln \pi(A_t|S_t,\theta)\right] \qquad \boxed{\theta_{t+1} = \theta_t + \alpha G_t \nabla_\theta \ln \pi(A_t|S_t,\theta)}$$

REINFORCE

- El algoritmo REINFORCE fue propuesto por (Williams, 1992).

Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning

Ronald J. Williams
College of Computer Science
Northeastern University
Boston, MA 02115

Appears in *Machine Learning, 8*, pp. 229-256, 1992.

Abstract

This article presents a general class of associative reinforcement learning algorithms for connectionist networks containing stochastic units. These algorithms, called REINFORCE algorithms, are shown to make weight adjustments in a direction that lies along the gradient of expected reinforcement in both immediate-reinforcement tasks and certain limited forms of delayed-reinforcement tasks, and they do this without explicitly computing gradient estimates or even storing information from which such estimates could be computed. Specific examples of such algorithms are presented, some of which bear a close relationship to certain existing algorithms while others are novel but potentially interesting in their own right. Also given are results that show how such algorithms can be naturally integrated with backpropagation. We close with a brief discussion of a number of additional issues surrounding the use of such algorithms, including what is known about their limiting behaviors as well as further considerations that might be used to help develop similar but potentially more powerful reinforcement learning algorithms.

REINFORCE

- Es un algoritmo Monte Carlo (no usa bootstrap).
- Espera al final del episodio para observar retornos completos y actualizar θ.
- Para aplicar el teorema PG se necesita:
  - ✓ una política parametrizada $\pi(a|s, \theta)$
  - ✓ muestreo de acciones A_t en cada instante t

REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for π_*

Input: a differentiable policy parameterization $\pi(a|s, \boldsymbol{\theta})$
Algorithm parameter: step size $\alpha > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \boldsymbol{\theta})$
  Loop for each step of the episode $t = 0, 1, \ldots, T-1$:
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$ $\qquad (G_t)$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \gamma^t G \nabla \ln \pi(A_t|S_t, \boldsymbol{\theta})$

REINFORCE CON BASELINE

- El problema de REINFORCE es que Gt (el retorno) es muy variable:
  - ✓ Puede ser muy grande o muy chico dependiendo de la trayectoria.
  - ✓ Esto genera ruido en el gradiente → dificulta aprender.
- Para reducir esa variabilidad, se introduce un término baseline b(St) = V(s) y se actualiza usando:

$$(G_t - b(S_t))\nabla_\theta \ln \pi(A_t|S_t, \theta)$$

  - ✓ Gt mide "qué tan bien salió la trayectoria completa".
  - ✓ b(St)b(St) actúa como un "promedio esperado" en ese estado StSt.
  - ✓ Entonces Gt−b(St) mide si lo que realmente pasó fue mejor o peor de lo que normalmente pasa.
- Resultado:
  - ✓ Si la acción fue mejor de lo esperado, la política se refuerza.
  - ✓ Si fue peor de lo esperado, la política se debilita.
  - ✓ La varianza se reduce, pero no se introduce sesgo (no cambia el valor esperado del gradient

REINFORCE CON BASELINE

- REINFORCE con baseline utiliza dos redes neuronales:
  - ✓ Una red para la política πθ.
  - ✓ Una red para el baseline Vw.
- Dos optimizadores separados: uno para θ, otro para w (aprende el valor esperado de un estado).

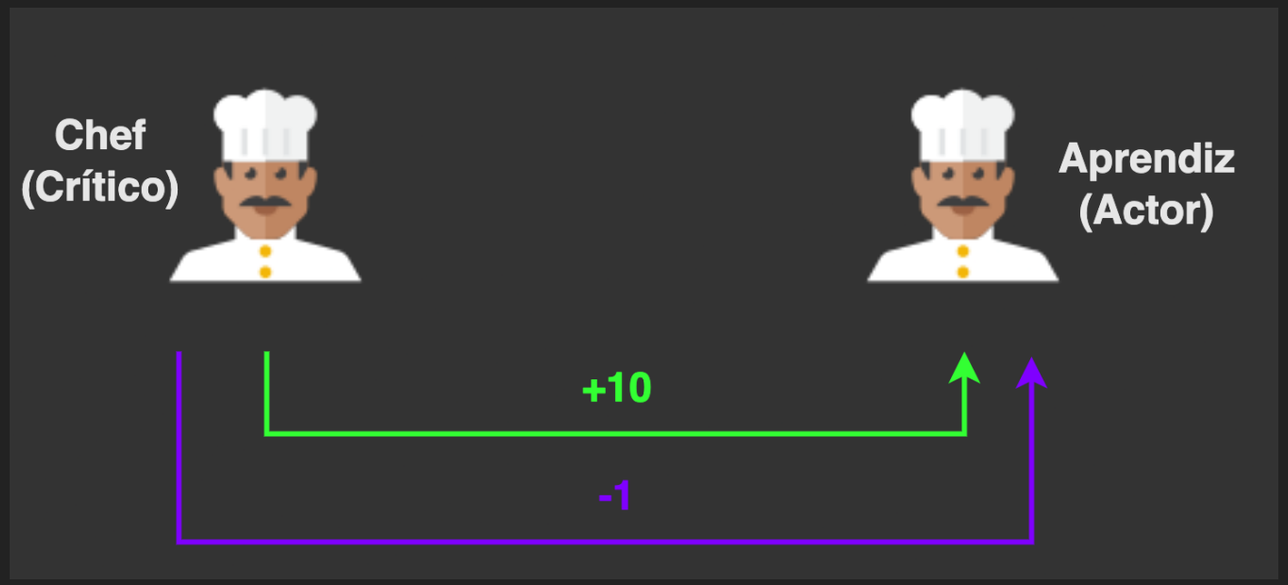
REINFORCE with Baseline (episodic), for estimating $\pi_{\boldsymbol{\theta}} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s, \boldsymbol{\theta})$
Input: a differentiable state-value function parameterization $\hat{v}(s,\mathbf{w})$
Algorithm parameters: step sizes $\alpha^{\boldsymbol{\theta}} > 0$, $\alpha^{\mathbf{w}} > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^{d}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \boldsymbol{\theta})$
  Loop for each step of the episode $t = 0, 1, \ldots, T-1$:
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$ $\qquad (G_t)$
    $\delta \leftarrow G - \hat{v}(S_t,\mathbf{w})$
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S_t,\mathbf{w})$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}} \gamma^t \delta \nabla \ln \pi(A_t|S_t, \boldsymbol{\theta})$

ACTOR-CRITIC

- Analogía: Un chef (crítico) y un aprendiz (actor).
- El aprendiz al inicio utilizará los utensilios incorrectos, quemará lo que intenta cocinar, utilizará proporciones de ingredientes inexactas, etc.
- El chef observará, corregirá o lo felicitará (otorga el valor de Q o recompensa según la acción).
- Ambos modelos aprenden (no solo el aprendiz).
- El crítico aprende a juzgar una acción comparando continuamente sus propias predicciones de valor con los resultados observados.



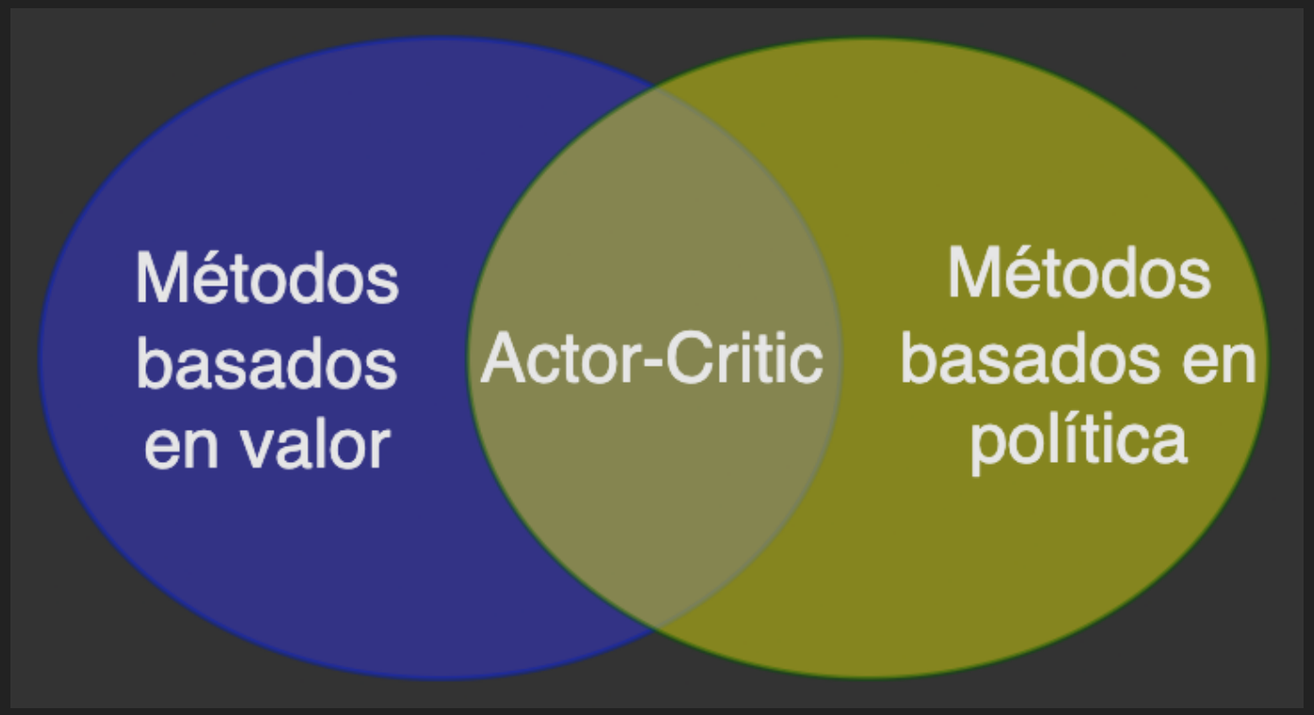
ACTOR-CRITIC

- Hasta ahora vimos 2 métodos de RL:
  - ✓ Basados en la función de valor (Q-Learning, DQN)
  - ✓ Basados en la política (REINFORCE)
- Ventajas:
  - ✓ Basados en valor -> mas estables y eficientes.
  - ✓ Basados en política -> mejores en espacios estocásticos y continuos.
- ¿conviene combinarlos para extraer lo mejor ambos métodos? SI

ACTOR-CRITIC

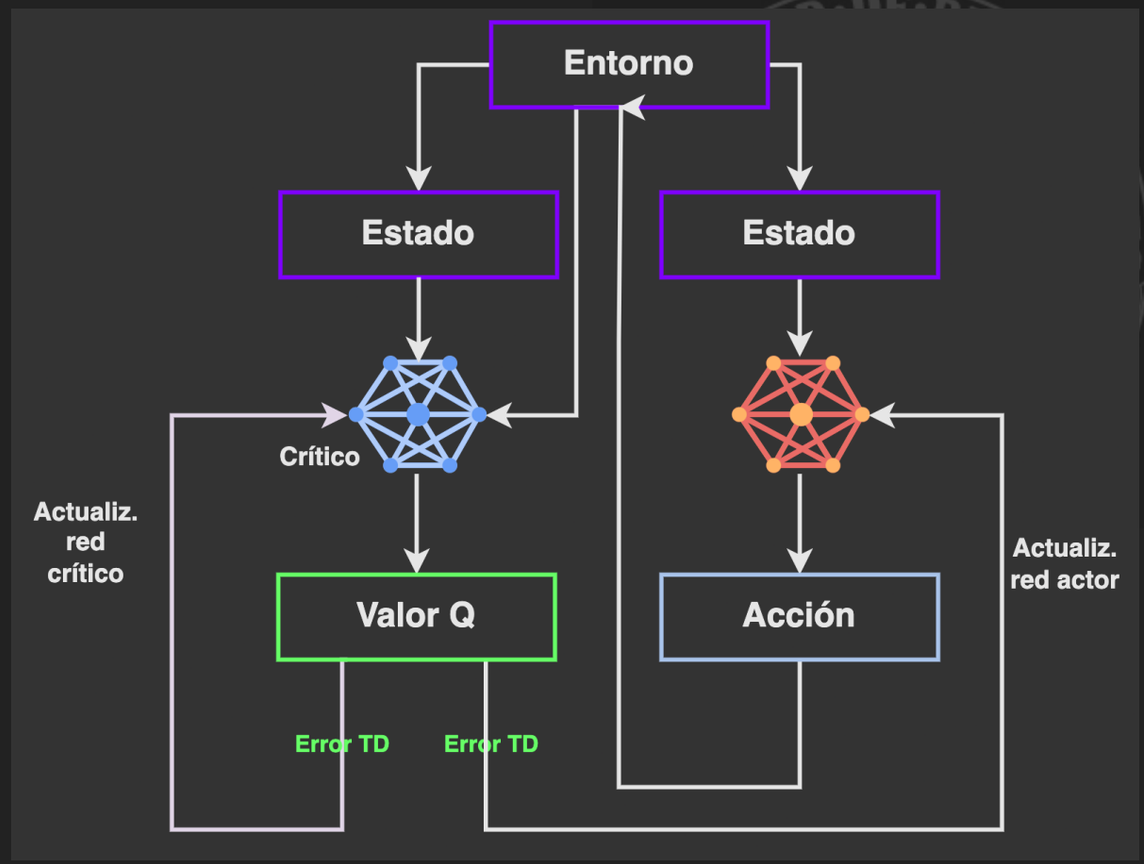
- El algoritmo Actor-Critic combina ambos métodos, esto es, emplea la función de valor y también la política para determinar la mejor acción en cada estado.

- A-C aprende explícitamente tanto la política (el actor) como una función de valor (el crítico), que a menudo estima los valores de estado.



ACTOR-CRITIC

- El modelo se divide en 2:
  - ✓ Una parte que determina la acción según el estado.
  - ✓ Y la otra parte determina el valor de Q de la acción.
- Para implementarlo se utilizan 2 redes neuronales:
  - ✓ Actor: es el que determina qué acción tomar en cada estado. Utiliza una función de aproximación que esta basada en la política.
  - ✓ Critico: es quien evalúa lo buena o lo mala que resultó la acción seleccionada para el estado actual. Utiliza una función de aproximación que esta basada en valor (Q).



ACTOR-CRITIC

- El algoritmo Actor-Critic combina ambos métodos, esto es, emplea la función de valor y también la política para determinar la mejor acción en cada estado.

- A-C aprende explícitamente tanto la política (el actor) como una función de valor (el crítico), que a menudo estima los valores de estado.

One-step Actor–Critic (episodic), for estimating $\pi_{\boldsymbol{\theta}} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s,\boldsymbol{\theta})$
Input: a differentiable state-value function parameterization $\hat{v}(s,\mathbf{w})$
Parameters: step sizes $\alpha^{\boldsymbol{\theta}} > 0$, $\alpha^{\mathbf{w}} > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^{d}$ (e.g., to $\mathbf{0}$)
Loop forever (for each episode):
  Initialize S (first state of episode)
  $I \leftarrow 1$
  Loop while S is not terminal (for each time step):
    $A \sim \pi(\cdot|S,\boldsymbol{\theta})$
    Take action A, observe S', R
    $\delta \leftarrow R + \gamma\hat{v}(S',\mathbf{w}) - \hat{v}(S,\mathbf{w})$ (if S' is terminal, then $\hat{v}(S',\mathbf{w}) \doteq 0$)
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}}\delta\nabla\hat{v}(S,\mathbf{w})$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}} I\delta\nabla\ln\pi(A|S,\boldsymbol{\theta})$
    $I \leftarrow \gamma I$
    $S \leftarrow S'$

REFERENCIAS BIBLIOGRÁFICAS Y WEB (I)

- Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine learning, 8, 229-256.
- R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.